Chapter

Introductory Chapter: An Overview of Using Virtual Instrumentation

Petru Adrian Cotfas, Daniel Tudor Cotfas and Horia Hedesiu

1. Introduction

1.1 Virtual instrument concept

We live in times when technology evolves very quickly in all domains, from medicine to automation or computer science. From measuring and controlling local systems and storing data in local PCs or company servers to measuring and controlling widespread distributed systems, cloud computing, and cloud storing data, the used technologies become more complex and require more effort to understand and use them.

Even though the concept of virtual instrumentation is not a new one, being introduced and used in the late seventies, it can be used with great success to facilitate the use and development of new technologies. One of the most widely used definitions states that virtual instrumentation is a combination of modular hardware and customizable software dedicated to making complex user-defined measurement, test, and control instruments. The instrument obtained based on the virtual instrumentation is called a virtual instrument (VI).

The evolution of computers and embedded systems also affects the evolution of virtual instrumentation due to the fact that the functionality of a device can be defined and modified by software and not totally by hardware. Based on this concept, the goal of a device can be modified by changing/updating only the software components (e.g., an oscilloscope that includes a data acquisition board and a software application can be changed in a spectral analyzer by adding a suitable data processing algorithm, or can be remotely controlled based on a network protocol and a suitable software application).

National Instruments, now NI, introduced the VI concept in the early eighties, having three components: Acquisition> > Processing > > Presentation. If the first component is more based on hardware, the last two are more based on software, which include data processing techniques (implemented in software) and data presentation facilities based on graphical user interfaces. By adding new data processing techniques, GUI components, or communication facilities, more complex and powerful instruments are obtained. On the other hand, sometimes not all the facilities of a traditional instrument (TI) are useful, being difficult to be used, and so, simpler instruments are recommended. Therefore, based on software, only the required facilities can be implemented to obtain a simple, user-friendly, and reliable instrument. Dedicated processors for specific tasks were an advantage for TI not so long ago, but now, the use of multicore processors and RT operating systems for VIs allows

this advantage to be mitigated or even overcome [1]. Meanwhile, other advantages for using VI appeared. The virtual instrument performance can be increased by upgrading the PC, or the used embedded system or parts of it (processors, RAM memory or storage memory, communication interfaces, etc.), or by upgrading the version of the used software. Thus, by replacing parts of the VI, not entirely, its performances can be increased. This advantage can be a part of the VI benefits versus TI related to the: price versus performance, maintenance, customization, and flexibility of usage depending on the user needs.

1.2 Graphical system design concept

A more recent concept has been used in instrumentation development, namely, Graphical System Design (GSD). According to Wikipedia [2], the GSD is "a modern approach to designing measurement and control systems that integrates system design software with commercially available off-the-shelf (COTS) hardware to dramatically simplify development." According to NI [3], the GSD is "a revolutionary approach to solving design challenges that blends intuitive graphical programming and flexible commercially available off-the-shelf (COTS) hardware to help engineers and scientists more efficiently design, prototype, and deploy embedded systems." Through this GSD concept, NI suggests using a unified software platform for the three stages of product implementation, namely, designing, prototyping, and deployment.

The unified software platform should allow the implementation of algorithms and mathematical models developed theoretically for the simulation of the desired system under the actions of stimuli and constraints that are obtained through theoretical approach or measured on real systems and stored in databases. This represents the design stage.

The second stage supposes building the system prototype at the laboratory level based on the same software platform and COTS hardware for project validation. The prototype is tested under action of the stimuli and constraints that are obtained at the laboratory level.

The third stage involves implementing the application on the final hardware platform used through appropriate adaptations and sizing. The final obtained system is tested under the stimuli and constraints that are obtained from the real working environment.

The time to implement a system based on a unified software platform is shortened because all parties involved have a good knowledge of it, and the platform also provides a consistent API that allows the control of modular and specific hardware.

2. NI LabVIEW as VI and GSD implementation environment

There are many solutions on the market that allow the implementation of the abovementioned concepts, VI and GSD. Such a unified software platform is a graphical programming platform, named NI LabVIEW (Laboratory Virtual Instrumentation Engineering Workbench). This platform offers modern programming techniques, such as graphical programming based on data flow paradigm, project-based programming, object-oriented programming, parallel and distributed programming optimized on modern multicore processors, and so forth. Being focused on the engineering field, there are many facilities, add-ons, and libraries that

DOI: http://dx.doi.org/10.5772/intechopen.113951

allow developing application for instruments control, monitor and testing systems, modeling and simulating, economical analyzing, digital signal and image processing, remote control, and web services. The last two mentioned facilities open widely the possibilities of developing applications in the field of Internet of Things, Industrial Internet of Things, and Cloud Computing.

NI LabVIEW is a graphical programming language that is different from other well-known programming languages. The big difference between these programming languages is that the programming in LabVIEW supposes the interconnection of graphical blocks with wires. The file extension of the LabVIEW applications is *.vi*, which comes from virtual instrument. The LabVIEW application consists of two windows: the first, called Front Panel (**Figure 1a**), represents the user interfaces, and the second, called Block Diagram (**Figure 1b**), represents the source code.

LabVIEW offers a large number of functions that allow implementing any kind of application in the fields of measuring, monitoring, testing and control, and more. LabVIEW includes advanced functions for data acquisition, signal processing, instrument communication, image processing, mathematics, data saving, control design and simulation, synchronization, report generation, and so on.

The advantages of LabVIEW programming for testing and monitoring applications include:

- easy and fast development of professional user interfaces (UI) through drag-and-drop UI objects that allow setting and displaying control values and results, respectively;

- connecting thousands of devices and instruments developed by NI or third parties using built-in functions;

- reusing already developed IP written in other programming languages such as Python, C/C++, MATLAB,. NET, VHDL, and more;

- "Programming like you think" based on the data flow paradigm.

In order to increase the flexibility, LabVIEW offers the possibility of interoperability with other applications or programming languages, such as SQL database based on *Database Connectivity Toolkit*,. NET environment based on. *NET functions,* MATLAB

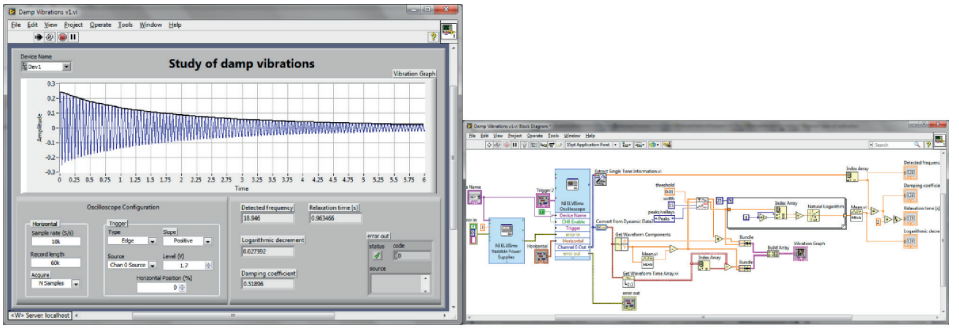


Figure 1.
The front panel (a) and block diagram (b) windows of a LabVIEW application.

based on the *MATLAB script nodes*, Python code based on *Python functions*, SPICE simulation based on *Multisim API*, DLLs through *Call Library Function Node*, and so forth (**Figure 2a**). On the other hand, LabVIEW allows implementing and using protocols for network or bus communications, such as Modbus, TCP/IP, UDP, FTP, HTTP, Serial, Bluetooth, DataSocket, Network Shared Variables, Network Streaming, and so on, or cloud communications based on SystemLink API (**Figure 2b**).

The concept of "programming like you think" correlates well with the data flow diagram used to graphically represent the algorithms that should be implemented in a programming language. Using graphical programming in LabVIEW, the data flow diagram is naturally transposed into code. This transposition is shown in **Figure 3**, where one can see how the initialization, measurement, processing, action, closure, and data saving steps are implemented. At the same time, parallel threads are easy to implement, through parts of the code that are not connected by threads.

Different programming architectures could be implemented in NI LabVIEW to increase the complexity of the developed applications. The developed application should respect the SMoRES concept, which supposes designing a code that has a high Scalability, Modularity, Reusability, Extensibility, and a solution implementation as Simple as possible. Such programming architectures are:

- State machine architecture—allows designing different states of the application that can be accessed any time based on the results of code execution.
- Producer-consumer architecture—dedicated to asynchronous multithreaded applications and where the consumer thread is executed only when the consumer data is available.
- Master-slave architecture—where the main thread always runs and triggers the slaves when to start executing code. The slave executes the code until it is completed and then waits for another trigger.
- Queued state machine and event-driven producer-consumer architecture—a combination of the other architectures used to implement a more complex application that allows handling multiple user-generated events stored in a queue and

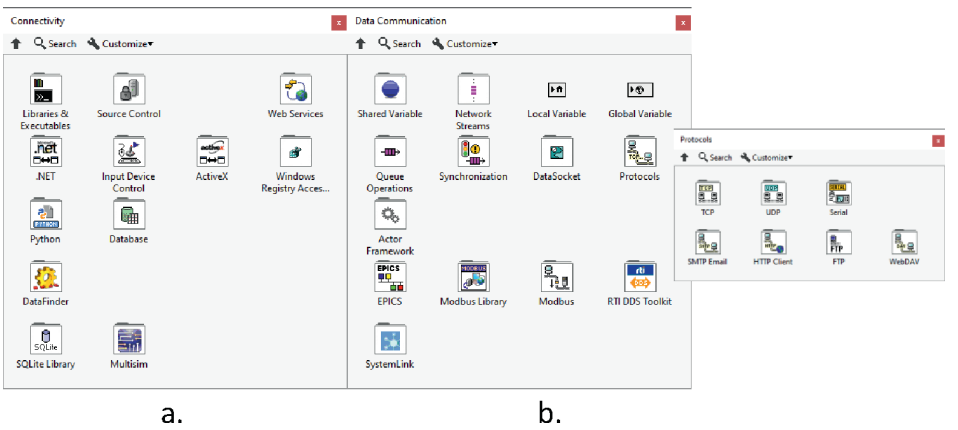


Figure 2.
a. LabVIEW connectivity pallet; b. LabVIEW data communication pallet.

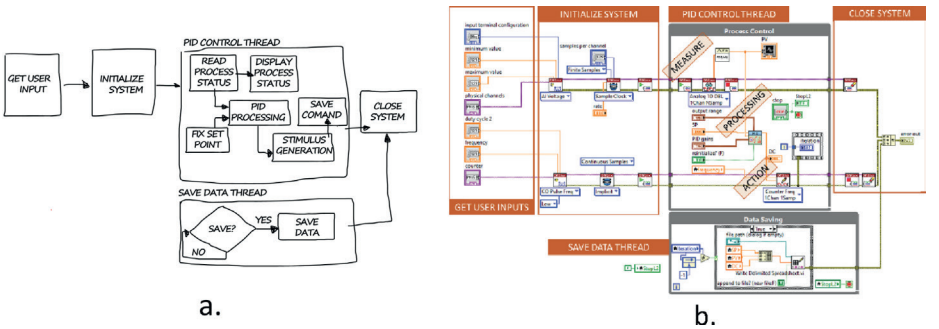


Figure 3.
The data flow diagram (a) transpose in NI LabVIEW code (b).

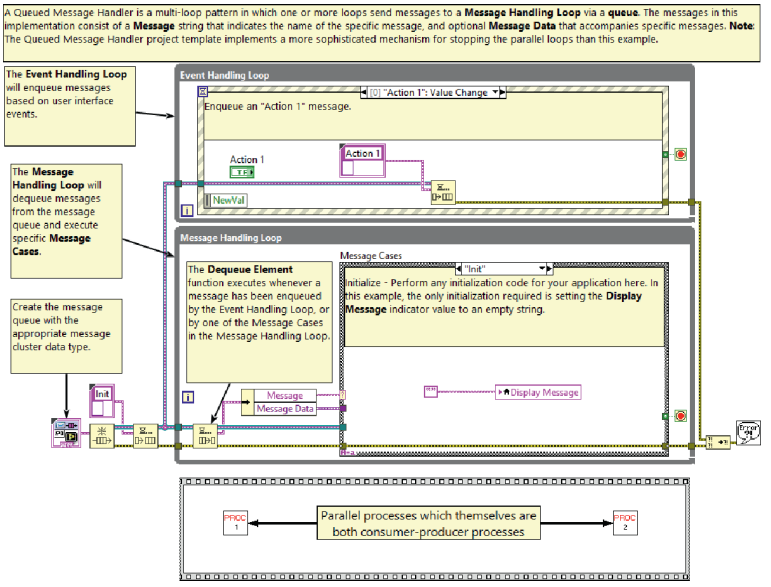


Figure 4.
LabVIEW example of queued state machine & event-driven producer-consumer architecture (from LabVIEW example library).

controls the sequence of states in the state machine consumer thread [4]. Other consumer threads can consume events generated by the state machine consumer, as can be seen in **Figure 4**.

LabVIEW can be used in industry, education, and research domains. A simple search into the scientific databases shows that LabVIEW is widely used in the research domain. Using "LabVIEW" as a search term, the IEEE Xplore database returns 6062 results divided into the following categories: conferences—5663, journals—342, magazines—40, books—13, early access articles—4, and ScienceDirect database returns 45,850 results, which are spread over the following domains: Engineering—17,092, Materials Science—9535, Physics and Astronomy—7925, Chemistry—7847, Energy—7168, Chemical Engineering—6650, Medicine and Dentistry—5422, Biochemistry, Genetics and Molecular Biology—3961, Neuroscience—3119, and Environmental Science—2375.

In the case of Scopus database, the search returned 36,561 results spread on the following domains: Engineering—23,373, Computer Science—13,193, Physics and Astronomy—8208, Mathematics—4789, Materials Science—4314, Energy—3812, Social Sciences—1968, Medicine—1673, Chemistry—1543, Chemical Engineering—1444, Biochemistry, Genetics and Molecular Biology—1335, Environmental Science—1194, Decision Sciences—1164, Agricultural and Biological Sciences—960, Earth and Planetary Sciences—815, Business, Management and Accounting—507, Multidisciplinary—371, Neuroscience—312, Health Professions—241, Pharmacology, Toxicology and Pharmaceutics—139, Arts and Humanities—125, Psychology—114, Immunology and Microbiology—92, Economics, Econometrics and Finance—80, Veterinary—31, Dentistry—30, and Nursing—26. The IntechOpen Publisher contains a list of over 20 chapters or books that include the term LabVIEW, since 2010. These results[1] spread on a very large spectrum of domains show that LabVIEW can be used with success in applications that suppose measurement, testing, simulation, and control, even if these fields are not among the engineering ones. Considering the above results, some domains of using LabVIEW are shortly presented below.

2.1 NI LabVIEW and embedded systems

The last development imposes the usage of open embedded systems, such as Arduino or Raspberry Pi platforms. Therefore, add-ons or APIs were developed for using these platforms with LabVIEW as it is shown in [5, 6]. Moreover, there is an add-on, named Arduino-Compatible Compiler for LabVIEW developed by the TSXperts company, that allows to compile and deploy VIs developed in LabVIEW on the Arduino platforms. For more complex applications based on embedded platforms, the NI LabVIEW RT and LabVIEW FPGA can be used [7, 8]. By implementing metaheuristic algorithms in FPGA, one can increase the power of the developed application [9]. In the case of complex applications based on embedded systems, NI considers one device that includes a processor, an FPGA, and a modular input/output (I/O). The architecture of this device is known as reconfigurable I/O (RIO) architecture. Using the NI LabVIEW FPGA module, the FPGA can be programmed, and using the built-in connectivity functions, the I/Os and processor can be accessed. With the LabVIEW RT module, a deterministic hard real-time application can be developed for control, data processing, and network connection with additional systems. The ease of writing FPGA code in LabVIEW FPGA is given by the fact that it is the same as writing normal LabVIEW code, using only the functions available for FPGA (**Figure 5**).

2.2 NI LabVIEW in energy

The energy demand of humanity is increasing, and therefore, the new energy sources or improving the efficiency of the existing ones are studied. Consequently, the models and simulations, testing and controlling, measuring, and monitoring of these sources should be done at industrial and educational levels. The LabVIEW Electrical Power Toolkit is one tool included in LabVIEW that offers the possibility to develop applications for electrical power and power quality measurements on hardware platforms like CompactRIO, CompactDAQ, and PXI. Characterization tools

[1] The results were obtained in 08.10.2023.

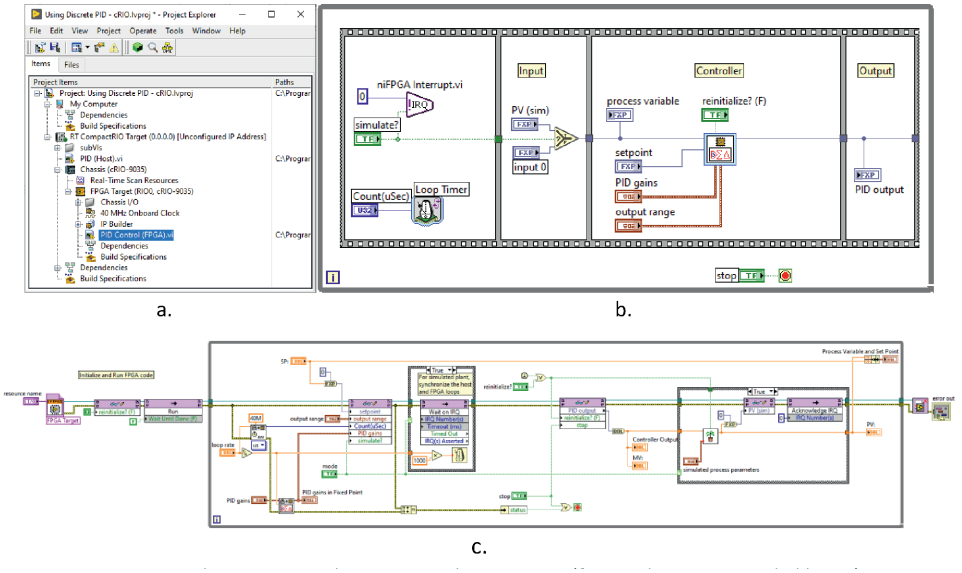


Figure 5.
NI LabVIEW RIO architecture implementation (from LabVIEW example library). a. LabVIEW project with PC, RT, and FPGA branches; b. LabVIEW FPGA implementation; c. LabVIEW RT implementation.

based on VI are often used in the field of photovoltaic systems [10–12], wind turbines [13–15], or hybrid systems [16–18].

2.3 NI LabVIEW in automotive industry

Test and control systems are one of the strengths of using LabVIEW, which is also found in the automotive industry. A very important step in V-shape control design that is also applied in the automotive industry is the hardware-in-the-loop (HIL) test step. HIL is a technique that involves the simulation of a real system through hardware and software that allows the physical connection and testing of a real device, device under test (DUT) (e.g., electronic control units, ECU). Real signals are passed between the DUT and the simulated systems, thereby tricking the DUT into working on the final product. There are many study cases that apply the HIL technique used in in-vehicle testing system [19], battery management system testing [20], or electrical vehicle management electronics testing [21].

2.4 LabVIEW in remote control

Lately, the remote (online) laboratories became an important component of the educational process. The remote labs became a vital component during the COVID pandemic. During the last two decades, many educators implemented different laboratories controlled remotely based on different software solutions and covering different education fields. Due to the facilities of hardware controls, interoperability with other software applications, and network communications, LabVIEW represents one of these solutions. Simple to complex solutions of remote laboratories are implemented and described in the specialty literature. An online experiment for eddy current magnetic brake study based on LabVIEW and NI myRIO is presented in [22], while in Ref. [23], a Digital Oscilloscope study is presented as part of instrumentation

and measurements course. Both solutions are based on a rapid but old technology called LabVIEW web publishing tool. More complex solutions were developed, such as the combination between the VISIR hardware platform and iLab remote control and management software platform offered by MIT [24]. A solution based on web programming techniques and web services under LabVIEW is presented in Ref. [25].

To increase the flexibility of web publishing of the LabVIEW developed application, the G Web Development Software was released by NI. This software package allows developing web-based user interfaces for applications developed in LabVIEW, Phyton, and/or C#, without the knowledge of classical web programming [26].

Another solution for remote control of testing and measuring instruments is the usage of the gRPC framework. This framework is an open source that was released by Google in 2015 and is based on Remote Procedure Call framework. gRPC is based on HTTP/2 and therefore is easy to integrate with a modern internet infrastructure. This framework is mainly dedicated to developing services, and there are eleven officially supported languages. There are several other languages that can support gRPC, and one of these is LabVIEW. The LabVIEW gRPC support can be found on GitHub (https://github.com/ni/grpc-labview).

3. Conclusion

Virtual instrumentation represents a strong and useful concept that is used in a large number of domains, from all engineering domains to medical or educational ones.

The combination of the modular hardware and the customizable software offers the possibility to develop simple and rapid applications but at the same time can be used for developing very complex applications applied to the most recent and modern domains.

One solution for implementing the virtual instrumentation concept is LabVIEW. Due to the graphical programming technique, this language can be rapidly learned and used by non-programming specialists. The NI LabVIEW applications can be developed by non-programming specialists due to the "programming like you think" approach and can cover programming of the RT and FPGA systems, HIL simulation systems, electrical power measurement systems based on dedicated LabVIEW toolkit, remote controlled systems based on web services and G Web Development Software, and numerous other systems and applications.

DOI: http://dx.doi.org/10.5772/intechopen.113951

Author details

Petru Adrian Cotfas[1*], Daniel Tudor Cotfas[1] and Horia Hedesiu[2]

1 Faculty of Electrical Engineering and Computer Science, Transilvania University of Brasov, Electronics and Computers Department, Romania

2 Faculty of Electrical Engineering, Technical University of Cluj-Napoca, Electrical Machines and Drives Department, Romania

*Address all correspondence to: pcotfas@unitbv.ro

IntechOpen

© 2024 The Author(s). Licensee IntechOpen. This chapter is distributed under the terms of the Creative Commons Attribution License (http://creativecommons.org/licenses/by/3.0), which permits unrestricted use, distribution, and reproduction in any medium, provided the original work is properly cited. CC BY

References

[1] Jennings R, De la Cueva F. LabVIEW Graphical Programming. 5th ed. New York: McGraw-Hill Education; 2020. Available from:https://www.accessengineeringlibrary.com/content/book/9781260135268

[2] Graphical system design. Available from: https://en.wikipedia.org/wiki/Graphical_system_design [Accessed: Oct. 03, 2023]

[3] Graphical system design basics: Accelerating development time and bringing embedded design to the masses. Available from: https://www.ni.com/en/shop/labview/graphical-system-design-basics--accelerating-development-time-an.html [Accessed: Oct. 03, 2023]

[4] Lukindo A. LabVIEW Queued State Machine Producer-Consumer Architecture. Canada: Mezintel Inc. Calgary AB; 2007. Available from: https://www.mezintel.com/blog/labview-queued-state-machine/ [Accessed: Oct. 03, 2023]

[5] Schwartz M, Manickum O. Programming Arduino with LabVIEW. Birmingham, UK: Packt Publishing; 2015

[6] Quang DT. LabVIEW and open embedded system [internet]. In: LabVIEW - A Flexible Environment for Modeling and Daily Laboratory Use. London: IntechOpen; 2021. DOI: 10.5772/intechopen.98271

[7] Kehtarnavaz N, Mahotra S. Digital Signal Processing Laboratory: LabVIEW-Based FPGA Implementation. Boca Raton, FL: Brown Walker Press; 2010

[8] Stratoudakis T. Introduction to LabVIEW FPGA for RF, Radar, and Electronic Warfare Applications. Norwood, MA: Artech House; 2021

[9] Ortiz A, Mendez E, Balderas D, Ponce P, Macias I, Molina A. Hardware implementation of metaheuristics through LabVIEW FPGA. Applied Soft Computing. 2021;**113**(Part A):29. DOI: 10.1016/j.asoc.2021.107908. Available form: https://0a10af1i5-y-https-doi-org.z.e-nformation.ro/

[10] Chouder A, Silvestre S, Taghezouit B, Karatepe E. Monitoring, modelling and simulation of PV systems using LabVIEW. Solar Energy. 2013;**91**:337-349. Available from: https://0a10aa2ix-y-https-doi-org.z.e-nformation.ro/10.1016/j.solener.2012.09.016

[11] Ahmed CB, Kassas M, Ahmed SE. LabVIEW based PV panel online characteristics and parameters estimation. Procedia Computer Science. 2015;**52**:876-882. DOI: 10.1016/j.procs.2015.05.145

[12] Cotfas PA, Cotfas DT. Design and implementation of RELab system to study the solar and wind energy. Measurement. 2016;**93**:94-101. DOI: 10.1016/j.measurement.2016.06.060

[13] Vukovic F, Milardic V, Milos D, Filipovic-Grcic B, Stipetic N, Franc B. Development and laboratory testing of a lightning current measurement system for wind turbines. Electric Power Systems Research. 2023;**223**:7. DOI: 10.1016/j.epsr.2023.109572

[14] Ogaili AAF, Jaber AA, Hamzah MN. Wind turbine blades fault diagnosis based on vibration dataset analysis. Data in Brief. 2023;**49**:6. DOI: 10.1016/j.dib.2023.109414

[15] Sudharsan GS, Vishnupriyan J, Anand KV, Chidambarathanu K. Wind Turbine Instrumentation System using Labview. In: Third International Conference on Intelligent Computing Instrumentation and Control Technologies (ICICICT), Kannur, India. IEEE. 2022. pp. 121-124. DOI: 10.1109/ICICICT54557.2022.9917754

[16] Kim M, Choi BY, Kang K-M, Lee H, Lee Y-S, Won C-Y. Energy monitoring system of AC/DC hybrid microgrid systems using LabVIEW. In: 23rd International Conference on Electrical Machines and Systems (ICEMS), Hamamatsu, Japan: IEEE; 2020. pp. 489-493. DOI: 10.23919/ICEMS50442.2020.9290836

[17] Aissou S, Rekioua D, Mezzai N, Rekioua T, Bacha S. Modeling and control of hybrid photovoltaic wind power system with battery storage. Energy Conversion and Management. 2015;**89**:615-625. DOI: 10.1016/j.enconman.2014.10.034

[18] Perz P, Malujda I, Wilczyński D, Tarkowski P. Methods of controlling a hybrid positioning system using LabVIEW. Procedia Engineering. 2017;**177**:339-346. DOI: 10.1016/j.proeng.2017.02.235

[19] Kumar AV. Creating a HIL simulation and in-vehicle test system using CompactRIO, FPGA, and LabVIEW, Tata Motors. Available from: https://www.ni.com/en/innovations/case-studies/19/creating-a-hil-simulation-and-in-vehicle-test-system-using-compactrio-fpga-and-labview.html [Accessed: Oct. 22, 2023]

[20] Hwang SU. Developing an HIL simulator for testing battery management system logic Sung-Up Hwang, Control Works. Available from: https://www.ni.com/en/innovations/case-studies/19/developing-an-hil-simulator-for-testing-battery-management-system-logic.html [Accessed: Oct. 22, 2023]

[21] Martínez J. Creating an Electric Vehicle HIL Test Using LabVIEW and CompactRIO Asoindel. Available from: https://www.ni.com/en/innovations/case-studies/19/creating-an-electric-vehicle-hil-test-using-labview-and-compactrio.html [Accessed: Oct. 22, 2023]

[22] Odema M, Adly I, Ghali HA. LabVIEW-based interactive remote experimentation implementation using NI myRIO. In: International Conference on Innovative Trends in Computer Engineering (ITCE), Aswan, Egypt. IEEE. 2019. pp. 214-218. DOI: 10.1109/ITCE.2019.8646602

[23] Hariton A, Zet C, Vremera E, Fosalau C. Remote laboratory – Study of digital oscilloscope, 2020. In: International Conference and Exposition on Electrical and Power Engineering (EPE), Iasi, Romania. IEEE. 2020. pp. 495-500. DOI: 10.1109/EPE50722.2020.9305551

[24] Zutin DG, Auer ME, Gustavsson I. A VISIR lab server for the iLab shared architecture. International Journal of Online Engineering. Barcelona, Spain: IATED 2016;7:14-17. DOI: 10.3991/ijoe.v7iS1.1754

[25] Cotfas PA, Cotfas DT, O.M. Machidon: Remote laboratories based on Labview web services. In: 8th International Conference on Education and New Learning Technologies - EDULEARN16, Barcelona, Spain. 2016. pp. 3683-3689. ISBN: 978-84-608-8860-4. ISSN: 2340-1117. DOI: 10.21125/edulearn.2016.1844

[26] Milan R. How to build a web UI for your LabVIEW-based test system. 2022. Available from: https://forums.ni.com/t5/Example-Code/How-to-Build-a-Web-UI-for-Your-LabVIEW-Based-Test-System/ta-p/4233446 [Accessed: Oct. 22, 2023]